

Giải Thích Chi Tiết Cách Hệ Thống Xử Lý Video Và Nhúng Tin

H.264/CAVLC: kênh dấu trailing-one, lịch nhúng có khóa và proof camera Groth16 gắn với video

Tài liệu kỹ thuật cho dự án

Cập nhật ngày 05/10/2026

Cập nhật 05/10/2026. Tài liệu mô tả hệ thống hiện tại: giao thức kênh **v3** (04/10/2026) và **proof camera gắn với video** (nguồn đối chiếu: `doc/he_thong_hoat_dong.md`, `README.md`, `demo/README.md`, `benchmark/NEW_BENCHMARKS.md` và mã nguồn). Những thay đổi chính:

- Chỉ còn **một lõi media native C++** (`native/`). Pipeline Python cũ (`parser/patcher` Python, bộ lọc vị trí nhiều tầng, file phụ vị trí, benchmark SEC1–SEC10) đã bị gỡ ngày 03/10/2026; các khái niệm “safety filter”, “patchability pruning”, “capacity funnel” trong bản cũ không còn tồn tại.
- Vị trí nhúng do **lịch HMAC có khóa** quyết định; trích xuất là **mù** (không cần video gốc, không cần file phụ).
- Giao thức kênh **v3**: hai khóa con HKDF-SHA256 (lịch, làm trắng), khung `[0x03][len 2B][payload]` **không có MAC** (overhead 3 byte; bản v2 cũ có thêm thẻ HMAC 16 byte, overhead 19 byte). Một giao thức theo đoạn IDR dùng chung cho file, job HTTP và luồng live.
- Proof Groth16 mới: mạch `camera_video.circom` (8737 constraint, 3 public input) chứng minh *một camera trong sổ đăng ký Merkle Poseidon đã xác nhận đúng video này và message này*; verifier chỉ cần root của sổ đăng ký, verification key và khóa giấu tin, **không** cần secret camera. Mạch SHA-256 cũ (63321 constraint) đã bị thay.
- Dịch vụ HTTP: job verify bắt buộc Groth16 với root sổ đăng ký và hash video của file tải lên; `run_all.py` 142/142 qua 12 phase; benchmark ghi ngày 04/10/2026.

1 Câu Trả Lời Ngắn Gọn Trước

Nếu nói thật ngắn, hệ thống làm việc như sau:

Nó không sửa pixel của video. Nó đặt lại giá trị của một số bit `trailing_ones_sign_flag` — bit dấu của hệ số trailing-one trong các khối residual CAVLC của slice IDR — tại những vị trí do một khóa bí mật chọn.

Một file H.264 được lõi native nhìn như một cấu trúc nhiều tầng:

`file .h264` → NAL units → IDR slices → macroblocks → residual blocks → bit dấu trailing-one

Mỗi khối residual có hệ số trailing-one cho đúng **một ứng viên**: bit dấu của trailing-one đầu tiên. Lật bit này không đổi độ dài mã, không đổi ngữ cảnh CAVLC, nên bitstream luôn

hợp lệ mà không cần kiểm tra “vả được hay không”. Việc còn lại là chọn ứng viên nào mang bit nào: hệ thống dùng HMAC theo khóa để xếp hạng ứng viên, rồi ghi một khung (đã làm trắng) vào các vị trí đứng đầu. Khung mang một proof Groth16 cho biết một camera đã đăng ký xác nhận đúng video và message đó. Bên nhận có cùng khóa giấu tin tự dựng lại đúng lịch từ video stego, rồi kiểm proof với root công khai của sổ đăng ký camera.

Ba lớp có trách nhiệm tách biệt:

Lớp	Thực hiện ở	Bảo đảm
Media	<code>native/src/cavlc_stream.cpp</code>	Bitstream sau nhúng vẫn hợp lệ, độ dài mã không đổi, decoder chuẩn giải được.
Kênh có khóa	<code>native/src/cavlc_stream.cpp</code> , <code>src/native_blind_contract.py</code>	Chỉ người có khóa giấu tin tìm được vị trí; sai khóa bị loại ngay ở bước kiểm header khung. Kênh <i>không</i> tự xác thực payload (khung v3 không có MAC).
Proof	<code>src/zk_proof.py</code> , <code>src/camera_proof.py</code> , <code>circuits/camera_video.circom</code>	Groth16: một camera trong sổ đăng ký (root công khai) xác nhận đúng video (hash video) và message; mọi sửa đổi video, message hay proof đều làm verify thất bại.

2 Codec, H.264 Và Vì Sao Hệ Thống Không Sửa Pixel

2.1 Codec là gì?

Codec là bộ mã hóa/giải mã video. Tên này đến từ:

`coder + decoder = codec`

Video thô rất lớn. Codec như H.264 nén video thành bitstream nhỏ hơn để lưu, truyền, hoặc phát lại. Khi phát video, decoder đọc bitstream đó và dựng lại các frame.

2.2 Sửa pixel và sửa bitstream khác nhau thế nào?

Một hệ đơn giản có thể làm như sau:

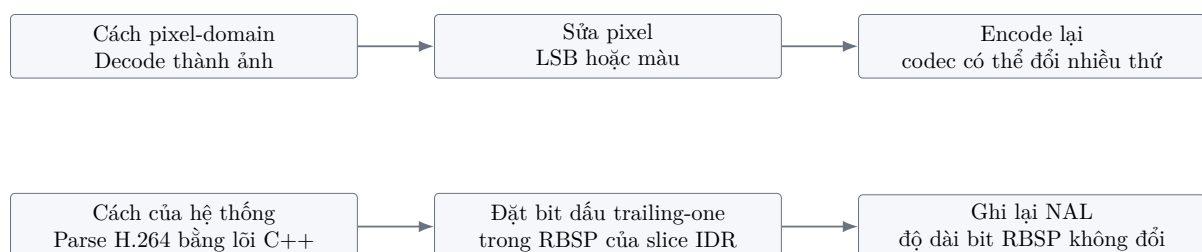
`decode video -> sửa pixel -> encode lại video`

Đó là cách làm ở **pixel domain**. Nó dễ hiểu nhưng có một vấn đề lớn: khi encode lại, codec có thể thay đổi rất nhiều thứ. Bit nhúng có thể mất, artifact có thể xuất hiện, và khó kiểm soát chính xác bitstream cuối cùng.

Hệ thống này đi theo hướng khác:

`parse H.264 (native) -> đặt bit dấu trong RBSP -> ghi lại NAL`

Đây là cách làm ở **compressed domain**. Không có bước encode lại: chỉ các bit dấu được chọn thay giá trị, mọi bit khác của file giữ nguyên (ngoại trừ emulation-prevention byte được tính lại, xem mục 4.3).



Hình 1: Khác biệt giữa sửa pixel và sửa dữ liệu nén trong H.264.

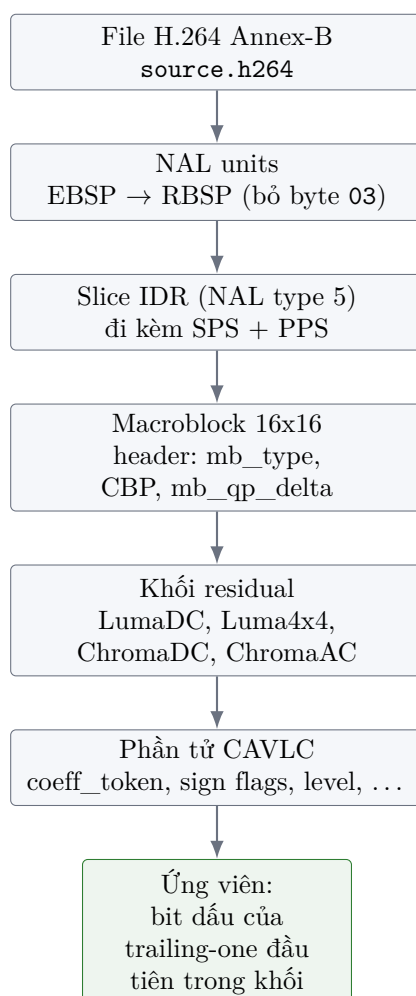
2.3 Video được chuẩn hóa về Baseline/CAVLC

Lỗi native chỉ hiểu một tập con H.264: Baseline, CAVLC, progressive, 4:2:0, một slice mỗi frame bắt đầu ở MB 0, không I_PCM, một SPS/PPS id. Vì vậy video được encode bằng FFmpeg/libx264 với tham số cố định (lệnh của demo, hàm `video_prepare` trong `demo/terminal_demo.py`):

Tham số	Lý do
<code>-profile:v baseline, cabac=0</code>	Parser native chỉ hỗ trợ CAVLC; CABAC không có kênh dấu độc lập như vậy.
<code>-g 1 / keyint=1</code>	Mọi frame là IDR. Chỉ IDR mang dữ liệu; GOP>1 vẫn chạy nhưng P-frame không có ứng viên.
<code>slices=1, threads=1</code>	Parser native chỉ nhận một slice mỗi frame.
<code>-bf 0</code>	Không B-frame.
<code>-qp 22</code>	Lượng tử cố định; x264 hạ QP của I-frame theo <code>ip_ratio=1.40</code> , nên slice có QP 19.

3 File H.264 Được Hệ Thống Bóc Ra Như Thế Nào?

Phần này là lỗi để nhìn rõ hệ thống. Khi nhận một file `source.h264`, lỗi native không nhìn nó như một chuỗi byte vô nghĩa. Nó bóc file theo các tầng sau.



Hình 2: Tầng xử lý từ file H.264 đến ứng viên nhúng.

Hàm native `serialize_cavlc_sign_candidate` gán cho mỗi ứng viên một định danh dạng chuỗi ASCII:

`nal_index:macroblock_address:category:block_index:rbsp_bit_offset`

Ví dụ trong `doc/he_thong_hoat_dong.md`:

`24:326:1:6:67801`

Nghĩa là:

- NAL thứ 24 (trong đầu vào đang phân tích),
- macroblock địa chỉ 326 trong slice,
- category 1 = Luma4x4 (0 = LumaDC, 2 = ChromaDC, 3 = ChromaAC),
- khối số 6 trong macroblock,
- bit dấu nằm ở offset bit 67801 của RBSP.

Định danh này chính là đầu vào của hàm score HMAC (mục 8). Trong giao thức theo đoạn, định danh được tính theo đầu vào của đoạn (SPS+PPS+IDR), không theo vị trí trong toàn file.

4 Ví Dụ Đầy Đủ: Một Video H.264 Được Parse Ra Gì?

Các giá trị dưới đây là **giá trị thật** lấy từ một lần chạy `demo/terminal_demo.py` trên `data/raw/foreman_cif.y4m` (CIF 352x288, 8 frame, QP 22, GOP 1), như được ghi trong `doc/he_thong_hoat_dong.md`. Có thể tự quan sát chúng bằng `zkstego_inspect` hoặc `demo`.

4.1 File H.264 ban đầu trông như thế nào?

Một file `.h264` thô là chuỗi NAL unit, mỗi NAL bắt đầu bằng start code 00 00 01 hoặc 00 00 00 01. Byte đầu tiên sau start code là NAL header: 1 bit `forbidden`, 2 bit `nal_ref_idc`, 5 bit `nal_unit_type`.

4.2 Kết quả parse NAL thật

Bảng NAL đầu của `source.h264` trong `demo`:

NAL	Offset	Start code	Header	Type	EBSP	RBSP	EPB
0	0	00000001	01100111	7 SPS	21	20	1
1	26	00000001	01101000	8 PPS	4	4	0
2	35	000001	00000110	6 SEI	568	568	0
3	607	000001	01100101	5 IDR	8797	8797	0

SPS/PPS cần để hiểu stream; SEI chứa chuỗi cấu hình của x264; chỉ NAL type 5 (slice IDR) mang dữ liệu nhúng.

4.3 EBSP, RBSP và emulation-prevention byte

Bên trong NAL, chuỗi 00 00 00, 00 00 01, 00 00 02, 00 00 03 không được xuất hiện. Encoder chèn byte 03 sau mỗi cặp 00 00 đứng trước byte ≤ 3 . Payload có byte chèn gọi là **EBSP**; bỏ các byte đó thu được **RBSP**. Ví dụ thật trong SPS: 04 40 00 00 03 00 40 00 $\rightarrow$ RBSP 04 40 00 00 00 40 00.

Mọi offset bit của parser native tính trên RBSP. Lật một bit RBSP có thể tạo hoặc xóa một mẫu 00 00 0x, nên khi ghi lại NAL, EPB được tính lại (`rbsp_to_esp`). Độ dài **bit RBSP** không đổi, nhưng độ dài **byte EBSP** có thể lệch vài byte.

4.4 SPS, PPS và slice header

Trích từ `trace_headers` của FFmpeg trên clip demo (vị trí bit tính từ đầu NAL, kể cả 8 bit NAL header):

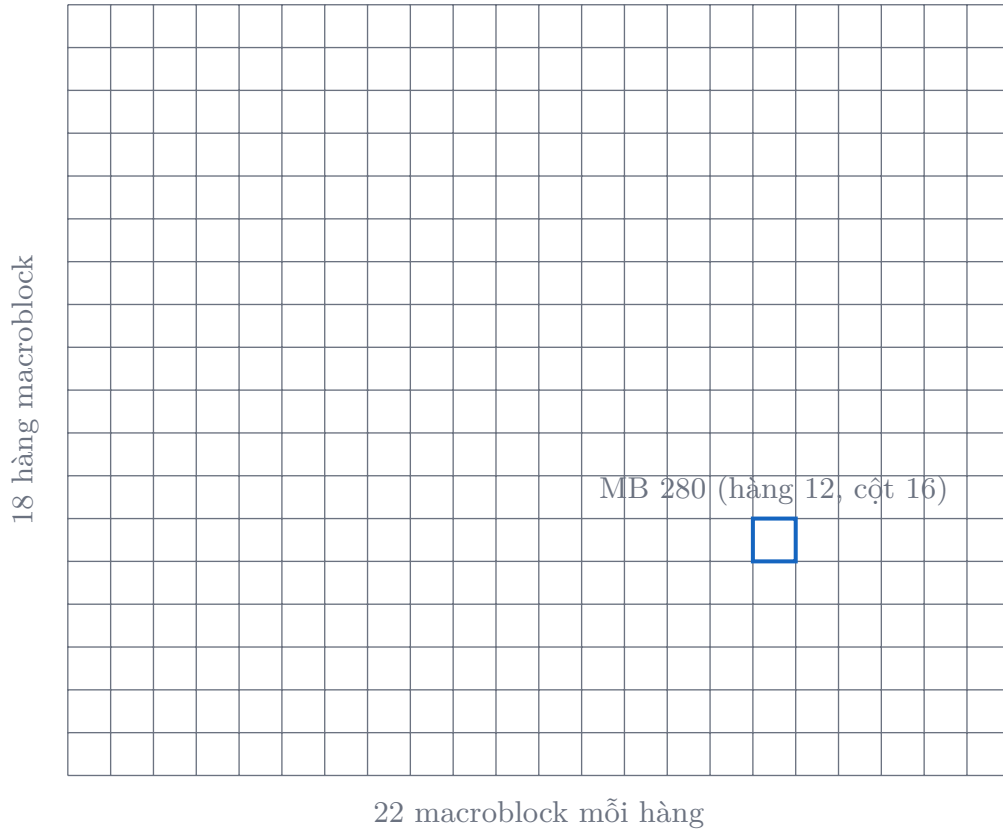
NAL	Trường	Giá trị
SPS	<code>profile_idc</code>	66 (Baseline)
SPS	<code>pic_width_in_mbs_minus1</code>	21 $\rightarrow$ 22 MB = 352 px
SPS	<code>pic_height_in_map_units_minus1</code>	17 $\rightarrow$ 18 MB = 288 px
PPS	<code>entropy_coding_mode_flag</code>	0 (CAVLC)
PPS	<code>pic_init_qp_minus26</code>	-4 $\rightarrow$ <code>pic_init_qp</code> 22
IDR	<code>slice_type</code>	7 (I)
IDR	<code>slice_qp_delta</code>	-3 $\rightarrow$ QP slice 19
IDR	<code>disable_deblocking_filter_idc</code>	0 (deblocking bật)

Slice header kết thúc ở bit 32 của NAL, tức bit 24 của RBSP; parser native báo `data_bit_offset = 24`. Demo kiểm tra hai con số này khớp nhau.

4.5 Từ IDR slice đến macroblock

Sau slice header là chuỗi macroblock theo thứ tự quét raster. Với CIF 352x288, mỗi frame có:

22 macroblock theo chiều ngang $\times$ 18 macroblock theo chiều dọc = 396
macroblock/frame.



Hình 3: Một frame CIF 352x288 được chia thành lưới 22x18 macroblock. Ô được tô là MB 280 ($280 = 12 \cdot 22 + 16$), macroblock dùng trong ví dụ bên dưới.

Header của một MB trong I-slice gồm `mb_type` (0 = I4x4, 1..24 = I16x16), 16 cờ/mode dự đoán Intra4x4 (nếu I4x4), `intra_chroma_pred_mode`, `coded_block_pattern` và `mb_qp_delta`. Ví dụ thật, MB 280 (đọc lại từng bit bởi `demo/h264_explain.py`):

Bit	Độ dài	Chuỗi bit	Phần tử = giá trị
48847	1	1	<code>mb_type</code> = 0 (I4x4)
48848	1	1	<code>prev_intra4x4_pred_mode_flag</code> [0] = 1
48850	1	0	<code>prev_intra4x4_pred_mode_flag</code> [2] = 0
48851	3	001	<code>rem_intra4x4_pred_mode</code> [2] = 1
...			
48888	1	1	<code>intra_chroma_pred_mode</code> = 0
48889	1	1	<code>coded_block_pattern</code> = 47
48890	1	1	<code>mb_qp_delta</code> = 0

Với CQP và `aq=0`, mọi `mb_qp_delta` = 0 nên QP mọi MB = 19; demo đối chiếu với bản đồ QP do decoder FFmpeg in ra (396/396 khớp).

4.6 Từ macroblock đến các khối 4x4

Một MB I4x4 có 16 khối luma 4x4, đánh số `luma4x4BlkIdx` theo zig-zag của khối 8x8 (không theo thứ tự raster):

0	1	4	5
2	3	6	7
8	9	12	13
10	11	14	15

Hình 4: Chỉ số `luma4x4BlkIdx` của 16 khối luma 4x4 trong một macroblock.

Ngoài 16 khối `Luma4x4`, residual của MB còn có `LumaDC` (chỉ với I16x16, 16 hệ số DC qua Hadamard; khi đó mỗi khối luma chỉ có 15 hệ số AC), `ChromaDC` (2x2 cho mỗi plane U/V) và `ChromaAC` (15 hệ số mỗi khối, 8 khối). Cả bốn loại đều có thể sinh ứng viên.

4.7 Một khối 4x4 sau khi parse trông như thế nào?

Ví dụ thật: frame #6, MB 280, khối `luma4x4BlkIdx` 2. Hệ số theo thứ tự zig-zag trước nhúng là `[2, -1, 0, ...]`; -1 là trailing-one. Ma trận hệ số:

2	-1	0	0
0	0	0	0
0	0	0	0
0	0	0	0

Với mỗi khối, `zkstego_inspect --macroblock NAL MB` xuất các trường (tên khóa JSON thật của công cụ):

Trường	Ý nghĩa
<code>category, index</code>	Loại khối và chỉ số khối trong MB.
<code>n_c</code>	Ngữ cảnh nC dùng để chọn bảng <code>coeff_token</code> .
<code>total_coeff, trailing_ones</code>	Số hệ số khác 0 và số trailing-one.
<code>coeff_token_bit, end_bit</code>	Vị trí bit bắt đầu <code>coeff_token</code> và bit kết thúc khối trong RBSP.
<code>sign_offsets</code>	Offset bit RBSP của các <code>trailing_ones_sign_flag</code> .
<code>coefficients_scan</code>	Hệ số theo thứ tự quét.

Ở bước này chưa có gì được nhúng. Parser chỉ liệt kê: khối nào có trailing-one và bit dấu đầu tiên nằm ở offset nào.

5 Residual Coefficient Là Gì?

H.264 thường không lưu trực tiếp từng pixel. Nó dự đoán nội dung của một khối từ các mẫu **đã tái tạo** bên trái/trên (dự đoán intra), rồi lưu phần sai khác giữa dự đoán và dữ liệu thật. Phần sai khác này gọi là **residual**. Sau biến đổi nguyên 4x4 (xấp xỉ DCT) và lượng tử hóa theo QP, residual trở thành các số nguyên gọi là **residual coefficients**, được quét zig-zag rồi mã hóa entropy bằng CAVLC.

Phía decoder làm ngược lại: giải lượng tử, biến đổi ngược, cộng với dự đoán, rồi áp deblocking cho cả frame. Hệ số trong ma trận **không phải pixel**: đổi một hệ số làm đổi cả khối residual 4x4 sau biến đổi ngược.

Hệ thống không sửa giá trị tuyệt đối của bất kỳ hệ số nào. Nó chỉ đổi **dấu** của một hệ số ± 1 nằm ở cuối dãy hệ số khác 0, vì dấu này được mã hóa bằng một bit riêng: `trailing_ones_sign_flag`.

6 CAVLC Và Trailing Ones

6.1 CAVLC là gì?

CAVLC là viết tắt của:

Context-Adaptive Variable Length Coding

Đây là cách H.264 Baseline mã hóa residual coefficients. Mỗi khối được mã thành 5 loại phần tử, đúng thứ tự (mục 9.2 của chuẩn):

Phần tử	Nội dung
<code>coeff_token</code>	Cặp (TotalCoeff, TrailingOnes); bảng VLC chọn theo <code>nC</code> .
<code>trailing_ones_sign_flag</code>	Mỗi trailing-one 1 bit dấu: 0 $\rightarrow$ +1, 1 $\rightarrow$ -1. Đây là kênh nhúng.
<code>level</code>	Các hệ số còn lại, giải theo thứ tự ngược scan (<code>level_prefix</code> + <code>level_suffix</code>).
<code>total_zeros</code>	Tổng số 0 đứng trước hệ số khác 0 cuối cùng.
<code>run_before</code>	Số 0 ngay trước mỗi hệ số.

`nC` là trung bình TotalCoeff của khối bên trái (`nA`) và bên trên (`nB`), làm tròn lên; ChromaDC dùng `nC` = -1. Vì `nC` của khối sau phụ thuộc TotalCoeff của khối trước, sửa *giá trị* hệ số bất kỳ có thể đổi bảng VLC và làm lệch toàn bộ phần còn lại của slice. Đó là lý do hệ thống chỉ chạm vào bit dấu.

6.2 Trailing ones là gì?

Trong CAVLC, tối đa ba hệ số cuối có trị tuyệt đối bằng 1 được mã hóa đặc biệt: chúng là **trailing ones** (T1). Độ lớn của chúng nằm sẵn trong `coeff_token`; chỉ dấu được ghi bằng một bit cố định.

Ví dụ thật từ demo (khối có `nC` = 4, bảng $4 \leq nC < 8$):

Bit	Độ dài	Chuỗi bit	Phần tử = giá trị
34636	4	1110	<code>coeff_token</code> = (1, 1): TotalCoeff 1, TrailingOnes 1
34640	1	1	<code>trailing_ones_sign_flag</code> [0] = 1 $\rightarrow$ hệ số -1 bit nhúng
34641	3	011	<code>total_zeros</code> = 1

Sau khi nhúng: bit 34640 = 0 $\rightarrow$ hệ số +1; mọi bit khác giữ nguyên. `demo/h264_explain.py` đọc lại đọc lập từng phần tử và kiểm tra việc đọc dừng đúng ở bit kết thúc do parser native báo.

6.3 Vì sao lật dấu trailing-one luôn an toàn về cú pháp

- **Không đổi độ dài mã:** chỉ một bit cố định thay giá trị; mọi phần tử sau giữ nguyên vị trí.
- **Không đổi ngữ cảnh:** TotalCoeff và TrailingOnes không đổi nên `nC` của các khối sau, bảng VLC được chọn và toàn bộ cú pháp phía sau giữ nguyên.
- **Thay đổi pixel nhỏ:** một hệ số đổi từ ± 1 sang ∓ 1 ($\Delta = \pm 2$ ở mức lượng tử).

Vì tính chất này đúng *theo cấu tạo*, lõi native không cần bước “thử sửa rồi mã hóa lại khối để xem có vá được không”. Mỗi ứng viên đều dùng được.

7 Ví Dụ Cụ Thể: Một Bit Đi Vào Một Vị Trí

Bit thực sự được ghi vào video không phải bit payload thô. Chuỗi biến đổi là:

$$\begin{aligned} \text{payload} &\rightarrow \text{khung v3} \rightarrow f_i \text{ (bit khung, MSB trước)} \\ &\rightarrow e_i = f_i \oplus k_i \rightarrow \text{trailing_ones_sign_flag} := e_i \end{aligned}$$

trong đó k_i là bit thứ i của keystream làm trắng (mục 10.2). Bit nhúng 0 $\rightarrow$ dấu +, bit nhúng 1 $\rightarrow$ dấu -, đúng quy ước của **trailing_ones_sign_flag**. Lỗi native **đặt** bit (không “lật” vô điều kiện): nếu dấu gốc đã trùng với e_i , bit đó không đổi.

Bảng minh họa dạng ký hiệu (ứng viên c_j là ứng viên đứng hạng j trong lịch của đoạn):

Bit khung i	f_i	k_i	e_i	Ứng viên	Dấu sau nhúng
0	0	1	1	c_0	-1
1	0	0	0	c_1	+1
2	1	1	0	c_2	+1
3	1	0	1	c_3	-1

Demo (bước “Bit nhúng nằm ở đâu”) in đúng chuỗi này cho một bit thật: định danh ứng viên, score HMAC, thứ hạng, chỉ số bit trong khung, trường mang bit (version/độ dài/format/mode/độ dài message/message/A.x/B.x/C.x/cờ y), byte RBSP, byte EBP (đếm EPB), offset trong file và hex trước/sau; rồi kiểm tra bit trong stego bằng bit khung XOR bit keystream.

8 Lịch Nhúng Có Khóa Theo Đoạn IDR

Vị trí nhận từng bit không cố định mà do khóa quyết định, để chỉ người có khóa đọc được. Đây là phần thay thế toàn bộ “pipeline chọn vị trí” của bản tài liệu cũ.

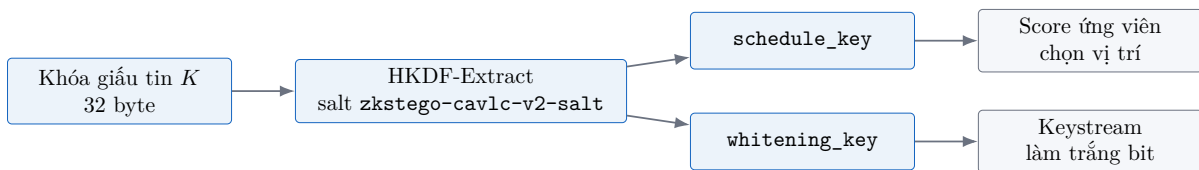
8.1 Khóa con HKDF (giao thức v3)

Hệ thống có **hai loại khóa tách biệt**: **khóa giấu tin** K (32 byte, người gửi và bên kiểm chứng cùng giữ; dùng để chọn vị trí và làm trắng bit) và **secret camera** s (chỉ camera giữ; dùng để tạo proof, mục 14). Trong các mục về kênh, “khóa” luôn là K .

Mỗi vai trò của kênh dùng một khóa con riêng, sinh bằng HKDF-SHA256 (RFC 5869) từ K . Từ giao thức v3 chỉ còn **hai** khóa con; nhân HKDF giữ nguyên như v2 nên lịch và dòng làm trắng không đổi:

```
PRK = HMAC-SHA256(key = "zkstego-cavlc-v2-salt", msg = K)
schedule_key = HKDF-Expand(PRK, "zkstego/cavlc/v2/schedule", 32)
whitening_key = HKDF-Expand(PRK, "zkstego/cavlc/v2/whitening", 32)
```

Lộ một khóa con (ví dụ khóa lịch) không làm lộ K hay khóa con còn lại. Ở đây HMAC chỉ đóng vai **hàm giả ngẫu nhiên có khóa** (chọn vị trí, sinh dòng làm trắng), không còn dùng để xác thực; khóa con thứ ba dùng cho thẻ khung của v2 đã bị bỏ.



Hình 5: Hai khóa con của giao thức v3 và vai trò của chúng.

8.2 Score và thứ tự ứng viên

$\text{score}(c) = \text{HMAC-SHA256}(\text{schedule_key}, \text{định danh ASCII của } c)$

Ứng viên được sắp xếp tăng dần theo (score, định danh); lấy N ứng viên đầu. Bit thứ i của khung (sau khi làm trắng) gán cho ứng viên thứ i .

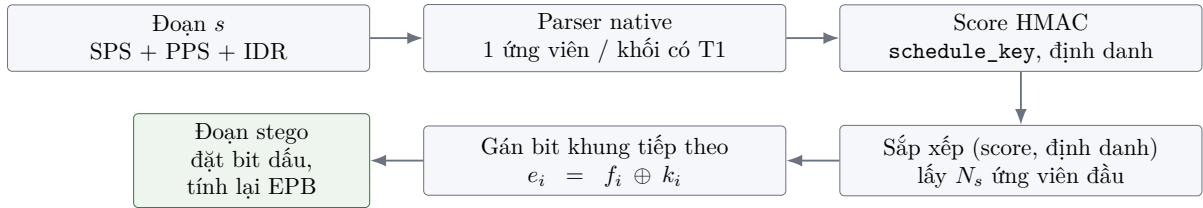
- Native: `score_keyed_cavlc_sign_candidate` và `select_keyed_cavlc_sign_candidates`.
- Python tham chiếu cùng ABI (có test vector chéo ngôn ngữ): `src/native_blind_contract.py`.

8.3 Một giao thức theo đoạn

Chỉ có một giao thức, **theo đoạn** (segment), dùng chung cho file, job HTTP và luồng live:

- mỗi đoạn = SPS + PPS + 1 IDR;
- đoạn s mang tối đa $N_s = \min(\text{max_bits_per_IDR}, \text{số ứng viên của } s)$ bit tiếp theo của khung, chọn theo lịch HMAC của chính đoạn đó;
- khung trải qua nhiều IDR liên tiếp; chỉ số bit khung đếm liên tục qua các đoạn;
- ở đoạn cuối, các vị trí trong lịch vượt quá cuối khung giữ nguyên dấu gốc.

Bên nhận phải dùng cùng `max_bits_per_IDR`, nếu không lịch lệch và trích sai. Dịch vụ dùng `ZK_STEGO_MAX_BITS_PER_IDR` (mặc định 64); demo dùng `--max-bits-per-idr` (mặc định 64).



Hình 6: Xử lý một đoạn IDR: liệt kê ứng viên, xếp hạng theo khóa, ghi bit.

8.4 Chất lượng ảnh

Lỗi native không có ngưỡng chất lượng trong bước nhúng; chất lượng được đo bên ngoài. Demo đo PSNR từng plane Y/U/V giữa `source.h264` và stego, in bản đồ $|\Delta Y|$. Đổi một hệ số lan truyền qua dự đoán intra sang các khối giải mã sau trong frame, bị deblocking làm mịn, và với $\text{GOP} > 1$ lan sang P-frame tham chiếu. Bộ benchmark `_new` chạy ngày 04/10/2026 (run 20261004T142448Z_1db00a, giao thức v3 + proof camera; 9 clip CIF $\times$ 3 độ phân giải 352x288, 640x480, 1280x960 = 27 case; QP 22, mọi frame là IDR, Baseline/CAVLC, 30 frame đầu; mọi proof gắn với video) ghi nhận: PSNR-Y thấp nhất cover $\rightarrow$ stego của từng case nằm trong khoảng 44,03–58,68 dB, 0/27 case dưới 40 dB; 26/30 frame bị thay đổi ở mọi case; kích thước stego bằng kích thước cover; không có bit dấu nào ngoài lịch bị đổi.

9 Capacity: Đếm Ứng Viên Theo Đoạn

Vì mọi ứng viên đều dùng được về mặt cú pháp, dung lượng không còn chia thành nhiều tầng lọc như bản cũ:

$$\text{capacity} = \sum_s \min(\text{max_bits_per_IDR}, \text{ứng viên}_s)$$

Một khung cần $(136 + \text{số byte message}) \times 8 \text{ bit}$: 4 byte header payload (format, mode, độ dài message 2 byte) + 129 byte proof + 3 byte overhead khung (1 version + 2 độ dài; không có thẻ MAC).

Đại lượng	Giá trị
Ứng viên của clip demo 8 frame CIF, QP 22, GOP 1	3 660
Ứng viên mỗi IDR, foreman QP 22	khoảng 450
Message 19 byte → khung	$3 + 4 + 19 + 129 = 155 \text{ byte} = 1\,240 \text{ bit}$ (khoảng 34% của 3 660)
Message 68 byte (benchmark 04/10/2026)	payload 201 byte → khung 204 byte = 1 632 bit → 26 IDR ở 64 bit/IDR
Clip E2E 300 frame CIF	300 đoạn IDR, 188 556 ứng viên; dung lượng ở cap 64 = 19 200 bit

Khoảng một nửa số vị trí trong lịch thực sự bị đổi dấu (phần còn lại đã sẵn trùng bit nhúng).

Với cap mặc định 64, clip ngắn có thể không đủ dung lượng ($8 \text{ IDR} \times 64 = 512 \text{ bit}$). Nếu không truyền `--max-bits-per-idr`, demo tự nâng cap lên giá trị nhỏ nhất đủ dung lượng và in lý do; nếu truyền mà thiếu thì dừng với gợi ý tăng `--frames`, giảm `--gop`, tăng cap hoặc rút ngắn message. `zkstego_inspect <video> --segments MAX_BITS` liệt kê ứng viên và dung lượng của từng đoạn (JSON `segments-1`); CLI `measure-live-capacity-stdin` in dung lượng dạng JSON từ stdin.

10 Payload Được Chuẩn Bị Như Thế Nào?

10.1 Payload

```
[format = 0x01 : 1 byte][mode : 1 byte][message_length : 2 byte BE][message][proof nén
                               : 129 byte]
```

- `mode = 0`: proof gắn với video (mặc định cho file và job HTTP).
- `mode = 1`: proof chỉ gắn với message (hash video thay bằng 32 byte 0), dành cho luồng live vì khi nhúng proof thì phần sau của video chưa tồn tại. Job verify từ chối chế độ này (lý do `video_binding_required`) trừ khi bật `ZK_STEGO_ALLOW_MESSAGE_ONLY_PROOFS=1`.

Hàm `pack_payload(mode, message, proof_bytes)` và `unpack_payload(blob)` nằm trong `src/zk_proof.py`; `unpack_payload` từ chối mọi bố cục khác (sai format, mode lạ, độ dài không khớp). Payload tối đa 4096 byte ở các lệnh stdin (giới hạn cấu hình; định dạng khung cho phép tới 65 535), nên message tối đa $4096 - 4 - 129 = 3963 \text{ byte}$.

10.2 Khung v3 và làm trắng

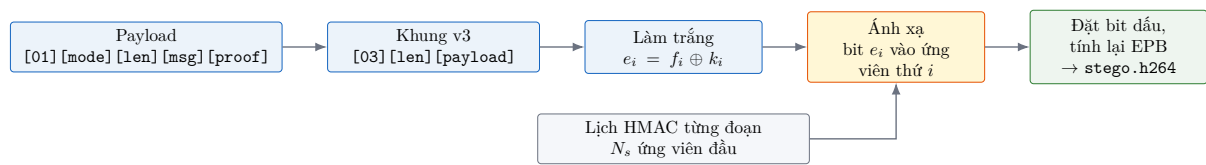
```
[version = 0x03 : 1 byte][payload_len : 2 byte BE][payload] (không có MAC)
keystream = HMAC(whitening_key, u64be(0)) || HMAC(whitening_key, u64be(1)) || ...
bit_nhung[i] = bit_khung[i] XOR bit_keystream[i]
```

Chỉ số i đánh trên toàn khung, kể cả khi khung trải nhiều IDR. Không làm trắng thì byte version cố định, trường độ dài nhỏ và cấu trúc proof làm thống kê dấu tại các vị trí được chọn bị lệch; sau khi XOR với dòng khóa giả ngẫu nhiên, các bit nhúng phân bố như ngẫu nhiên đối với người không có khóa. Dòng làm trắng không có nonce: cùng khóa K thì mọi lần nhúng dùng lại cùng keystream.

Vì sao bỏ thẻ HMAC. Khung v2 (lịch sử) mang thêm một thẻ HMAC 16 byte trên header và payload, với một khóa con riêng; overhead 19 byte. Khi Groth16 verify đã là bắt buộc, thẻ này trùng việc: sửa message hay proof thì proof không còn hợp lệ. Thẻ tiết kiệm 128 bit (2 IDR ở 64 bit/IDR). Hệ quả: kênh không còn tự phát hiện bit payload bị sửa, nên **chỉ bước verify Groth16** mới xác thực payload; lệnh/job extract trả payload kèm "verified": false. Giao thức v3 không đọc được stego tạo bởi v2 hay v1 (byte version khác).

Thành phần	Kích thước	Ghi chú
Version	1 byte	0x03
Độ dài payload	2 byte	Big-endian
Format	1 byte	0x01
Mode	1 byte	0 = gán video, 1 = chỉ message
Độ dài message	2 byte	Big-endian
Message	m byte	UTF-8
Proof nén	129 byte	Groth16 BN254
Tổng	$136 + m$ byte	$m = 19 \Rightarrow 155 \text{ byte} = 1\,240 \text{ bit}$

11 Payload Được Nhúng Vào Video Như Thế Nào?



Hình 7: Từ payload đến video stego: khung v3, làm trắng, lịch có khóa.

Lệnh native (khóa không bao giờ đi qua tham số dòng lệnh):

```
zkstego_blind_bits embed-stream-auth-stdin in.h264 out.h264 <max_bits_per_IDR>
```

stdin dòng 1 = khóa 64 ký tự hex, dòng 2 = payload hex. Mã thoát 2 cho mọi lỗi. Ngoài ra, `embed-live-auth-stdin` và `extract-live-auth-stdin` làm việc y hệt nhưng video vào/ra qua stdin/stdout (dùng cho WebSocket).

11.1 Ảnh hưởng lên một khối: ví dụ thật

Khối `luma4x4BlkIdx 2` của MB 280, frame #6 (mode Intra4x4 suy ra là 1, Horizontal; QP 19). Lật dấu trailing-one:

Hệ số trước nhúng	Hành động	Hệ số sau nhúng																																
<table><tr><td>2</td><td>-1</td><td>0</td><td>0</td></tr><tr><td>0</td><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>0</td><td>0</td><td>0</td></tr></table>	2	-1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	trailing_ones_sign_flag 1 → 0 hệ số -1 → +1	<table><tr><td>2</td><td>+1</td><td>0</td><td>0</td></tr><tr><td>0</td><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>0</td><td>0</td><td>0</td></tr></table>	2	+1	0	0	0	0	0	0	0	0	0	0	0	0	0	0
2	-1	0	0																															
0	0	0	0																															
0	0	0	0																															
0	0	0	0																															
2	+1	0	0																															
0	0	0	0																															
0	0	0	0																															
0	0	0	0																															

Hình 8: Ví dụ thật: nhúng một bit bằng cách đổi dấu một hệ số trailing-one.

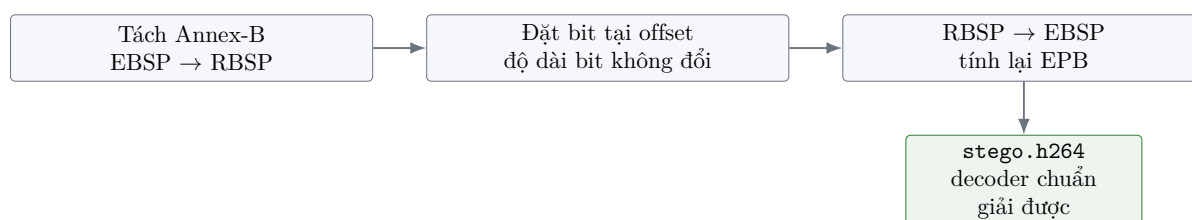
Bước (mỗi hàng của khối)	Trước	Sau
Giải lượng tử, hàng 0	[176, -112, 0, 0]	[176, 112, 0, 0]
Residual sau biến đổi ngược	[1, 2, 4, 5]	[5, 4, 2, 1]
Dự đoán Horizontal	cột trái = 83	
Tái tạo (trước deblocking)	[84, 85, 87, 88]	

Giá trị tái tạo “trước” khớp ảnh FFmpeg `-skip_loop_filter` all từng mẫu. Sau nhúng, pixel của khối đổi [6, 4, 0, -2] mỗi hàng = Δ dự đoán [2, 2, 2, 2] + Δ residual [4, 2, -2, -4]. Δ dự đoán khác 0 vì cột mẫu bên trái (thuộc MB 279) đã đổi do một lần lật dấu giải mã trước đó: ảnh hưởng **lan truyền** qua dự đoán intra.

12 Ghi Lại Bitstream: Vì Sao Video Sau Nhúng Vẫn Là H.264 Hợp Lệ?

Bản cũ có bước “reconstruction” mã hóa lại khối CAVLC và đếm lại số vị trí “sống sót”. Lỗi hiện tại không cần bước đó:

1. Tách Annex-B thành NAL, chuyển NAL cần sửa sang RBSP.
2. Với mỗi vị trí trong lịch, **đặt** bit tại offset `rbsp_bit_offset` thành bit nhúng; độ dài RBSP không đổi. Hàm native: `apply_fixed_length_patches`.
3. Chuyển RBSP về Ebsp (chèn lại EPB) và ghép lại Annex-B.



Hình 9: Ghi lại bitstream: không mã hóa lại, chỉ đặt bit dấu rồi tính lại EPB.

Bằng chứng “vẫn hợp lệ” đến từ kiểm thử, không phải từ bộ đếm sau nhúng: phase 3 (`test_native_cli_fixture.py`) nhúng/trích trên fixture 300 frame với decode nghiêm ngặt; demo giải mã stego bằng FFmpeg và kiểm số frame, ánh xạ 1 slice/frame, danh tính ứng viên, bit và metadata không đổi giữa trước/sau.

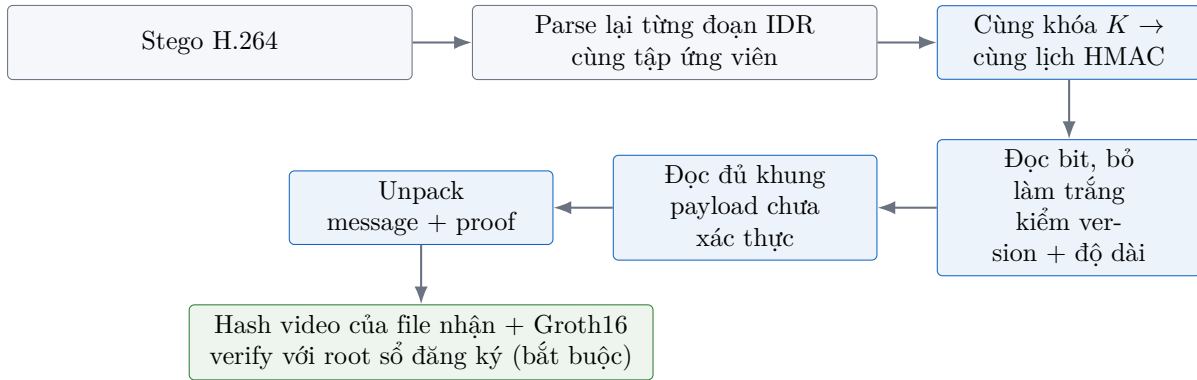
12.1 Đầu ra

Đầu ra của lệnh nhúng chỉ là **một file stego.h264**. Không có file phụ chứa vị trí. Khi chạy demo, thư mục `demo/runs/terminal_<thời gian>_<ngẫu nhiên>/` chứa thêm các artifact để quan sát (không cần cho việc trích xuất):

File	Nội dung
source.h264, stego.h264	Video trước/sau nhúng.
schedule.json	Mọi vị trí được dùng: đoạn, bit khung, ID trong đoạn, ID file, score, bit trước/đích.
trace_before.json, trace_after.json	Mọi NAL, slice, ứng viên, bit dấu, hệ số.
segments_before.json, segments_after.json	Input lịch theo đoạn (segments-1).
proof.json, proof.bin, public_signals.json, camera_registry.json	Proof, 3 public input và sổ đăng ký camera của demo.
transcript.txt, report.json	Thuyết minh (secret bị che), kết quả kiểm tra, timing, PSNR.

13 Trích Xuất Payload Diễn Ra Như Thế Nào?

Trích xuất là **mù**: bên nhận chỉ cần video stego và khóa giấu tin K , không cần video gốc, không cần file phụ. Để *tin* payload, bên nhận cần thêm root của sổ đăng ký camera và verification key (mục 14); không cần secret camera.



Hình 10: Pipeline trích xuất mù: dựng lại lịch từ stego và khóa, đọc khung, rồi verify proof gắn với video.

1. Parse lại các IDR của video stego, liệt kê ứng viên. Tập ứng viên giống hệt phía gửi vì lật dấu không đổi TotalCoeff/TrailingOnes, tức không đổi cấu trúc.
2. Dùng cùng khóa K sinh lại các khóa con, tính score, sắp xếp, đọc bit tại các vị trí đã chọn (cùng `max_bits_per_IDR`).
3. Bỏ làm trắng 24 bit đầu $\rightarrow$ kiểm version = 3 và độ dài $\leq$ giới hạn $\rightarrow$ biết tổng số bit cần đọc. Sai khóa thì byte version sau khi bỏ làm trắng gần như ngẫu nhiên nên bị từ chối ngay (lọt qua cả version lẫn độ dài với xác suất cỡ $\frac{1}{256} \cdot \frac{\text{giới hạn}}{65536}$; các ca sót lại hỏng ở bước giải nén proof hoặc Groth16).
4. Đọc đủ khung và bỏ làm trắng $\rightarrow$ payload. Payload này **chưa được xác thực** ("verified": false) cho tới khi proof Groth16 verify.

Lệnh native:

```
zkstego_blind_bits extract-stream-auth stego.h264 - <max_payload> <max_bits_per_IDR>
```

stdin: khóa hex; stdout: payload hex (chưa xác thực). Các thông báo lỗi native (đều thoát mã 2):

Thông báo	Khi nào
CAVLC frame/stream version is invalid	Byte đầu sau khi bỏ làm trắng khác 0x03 (thường do sai khóa, hoặc stego v2).
CAVLC stream length exceeds configured maximum, CAVLC frame length is invalid	Trường độ dài vượt giới hạn hoặc không khớp.
blind schedule capacity is insufficient	Lịch cần nhiều bit hơn số ứng viên của đoạn.
IDR stream ended before payload was complete, CAVLC stream is incomplete	Hết video trước khi đọc đủ khung.

Tìm thấy khung chỉ có nghĩa là có dữ liệu ở đúng vị trí theo khóa; **không** chứng minh payload nguyên vẹn hay đến từ camera hợp lệ. Việc đó thuộc về proof Groth16: CLI native không kiểm proof, nên quy tắc “proof bắt buộc” do `src.camera_proof.verify_payload`, demo và job `/api/v1/jobs/verify` thực thi ngay sau trích xuất.

14 Cơ Chế ZKP: Proof Được Sinh Từ Đâu Và Sinh Như Thế Nào?

ZKP không chọn vị trí nhúng trong video. ZKP tạo ra một proof mật mã chứng minh rằng **một camera nằm trong sổ đăng ký (gốc Merkle công khai) đã xác nhận đúng video này và message này**, mà bên kiểm chứng không cần secret của camera và không biết camera nào. Proof được nén thành 129 byte, đóng gói vào payload; đối với lõi media nó chỉ là dữ liệu cần giấu và đọc lại chính xác.

Hai khóa được tách hẳn:

Khóa	Ai giữ	Dùng để
Khóa giấu tin K (32 byte)	Người gửi và bên kiểm chứng	Chọn vị trí nhúng, làm trắng bit, định vị bit carrier của hash video.
Secret camera s (phần tử trường BN254)	Chỉ camera	Tạo proof; sổ đăng ký chỉ lưu $pk = \text{Poseidon}(s)$.

14.1 Sổ đăng ký camera

`src/camera_registry.py`: cây Merkle nhị phân độ sâu 16 (tối đa 65536 camera), nút $\text{Poseidon}(\text{trái}, \text{phải})$, lá trống = 0, lá $i = pk_i = \text{Poseidon}(s_i)$. Poseidon được cài bằng Python (`src/poseidon.py`, hằng số của circomlib) và đã được đối chiếu với vector chuẩn của circomlibjs và với witness của chính mạch. Bên kiểm chứng chỉ tin **root** của cây.

14.2 Hash video và binding

Lệnh `zkstego_blind_bits video-digest <video> <frame_bits> <cap>` (đọc khóa K qua stdin; bản tham chiếu `src/video_binding.py`) tính:

```
frame_bits = 8 * (3 + số byte payload) (biết trước khi tạo proof)
carrier = đúng frame_bits vị trí đầu mà lịch theo khóa K sẽ ghi khung vào
digest = SHA256("zkstego/video-digest/v1kern.03em| u32(frame_bits) || u32(cap) ||
    với mọi NAL theo thứ tự: u32(1 + len(rbsp')) || header || rbsp')
rbsp' = RBSP của NAL (đã bỏ EPB), CHỈ các bit carrier được đặt về 0
binding = SHA256("zkstego/proof-binding/v1kern.03em| mode || digest || SHA256(message))
bindingHi, bindingLo = hai nửa 128 bit của binding (public input)
```

Nhúng chỉ đổi đúng các bit carrier (EPB có thể thêm/bớt nhưng không ảnh hưởng RBSP), nên **digest của cover = digest của stego**. Mọi thay đổi khác — video khác, cắt hay thêm

frame, sửa một bit ở frame sau, lật một bit dấu không mang khung, mã hóa lại — làm digest đổi và proof không còn đúng. Lật một bit carrier thì digest giữ nguyên nhưng payload trích ra đổi, nên proof cũng bị từ chối. Vị trí carrier phụ thuộc khóa K , nên chỉ người có khóa mới tính được digest (bên kiểm chứng vốn đã cần K để trích). Với `mode = 1` (luồng live), digest được thay bằng 32 byte 0.

14.3 Mệnh đề mà proof chứng minh

Mạch CameraVideo trong `circuits/camera_video.circom`:

Ràng buộc	Ý nghĩa
$pk = \text{Poseidon}(s)$	Khóa công khai của camera suy từ secret.
Đường Merkle từ pk dẫn tới root	Camera nằm trong sổ đăng ký.
$\text{pathIndices}[i] \cdot (\text{pathIndices}[i] - 1) = 0$	Chỉ số đường đi là bit.
$\text{bindingHi}^2, \text{bindingLo}^2$	Gắn binding vào R1CS.

Mạch có **8 737 constraint** và **3 public input**. Proof tạo cho một binding (video + message) hay một root khác đều không verify được. (Mạch SHA-256 trước đây, `SHA256(SHA256(m) || secret)`, có 63 321 constraint và buộc verifier giữ secret; nó đã bị thay ngày 04/10/2026, và proof tạo bằng khóa của mạch cũ không verify được với khóa hiện tại.)

14.4 Public input và private input

Loại input	Tên	Ý nghĩa
Public	<code>root</code>	Root của sổ đăng ký camera mà verifier tin.
Public	<code>bindingHi</code>	128 bit cao của binding.
Public	<code>bindingLo</code>	128 bit thấp của binding.
Private	<code>secret</code>	Secret camera s .
Private	<code>siblings[16]</code>	Các nút anh em trên đường Merkle.
Private	<code>pathIndices[16]</code>	Hướng trái/phải ở mỗi tầng.

14.5 Dữ liệu trước khi tạo proof

Hàm `build_payload(bridge, registry, camera_secret, message, native_cli=..., cover=...)` trong `src/camera_proof.py`:

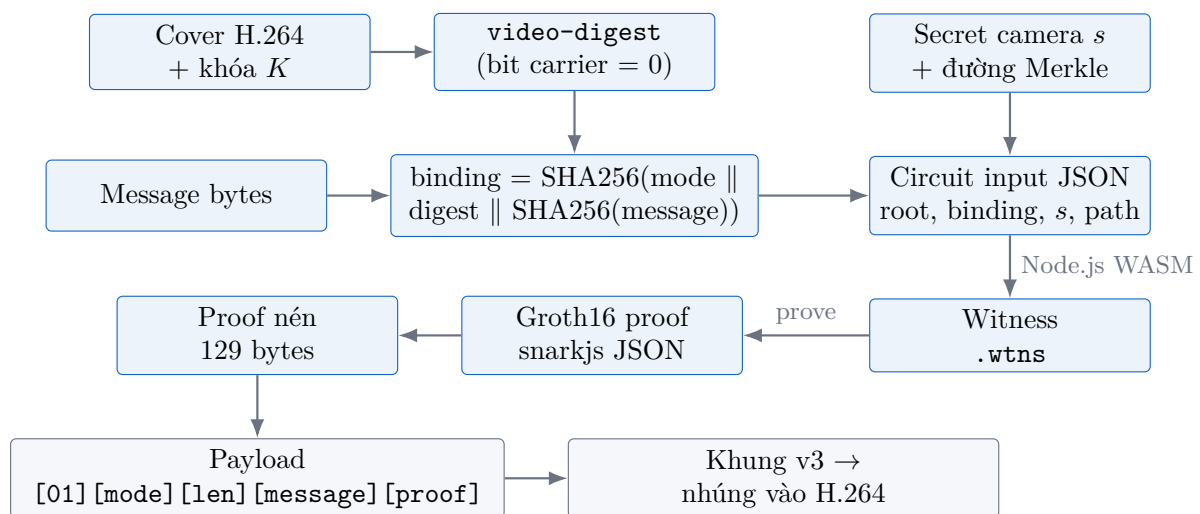
Bước	Giá trị tạo ra
1	<code>frame_bits = 8 * (3 + 4 + len(message) + 129)</code> .
2	<code>video-digest</code> của <code>cover</code> theo khóa K và cap.
3	<code>binding</code> từ <code>mode</code> , <code>digest</code> và <code>SHA256(message)</code> ; tách <code>bindingHi/bindingLo</code> .
4	Lấy đường Merkle của $pk = \text{Poseidon}(s)$ trong sổ đăng ký; dựng input JSON.
5	Groth16 prove, nén proof, <code>pack_payload</code> → <code>payload</code> .

14.6 Workflow sinh proof và setup

Hệ thống dùng `snarkjs` và Groth16 (`CameraProofBridge` trong `src/zk_proof.py`). Các artifact nằm trong `circuits/build/` (bị git-ignore):

Artifact	Vai trò
camera_video_js/camera_video.wasm	Chạy circuit để tạo witness từ input.
camera_video_js/generate_witness.js	Script Node.js sinh witness.
camera_video.zkey	Proving key dùng để tạo Groth16 proof.
camera_video_vkey.json	Verification key dùng để verify proof.
camera_video_setup.json	Transcript của nghi thức setup.

Setup (py -3.12 -m src.zk_setup, hoặc demo/terminal_demo.py --setup): Phase 1 là Powers of Tau công khai của Hermez (2^{16} , powersOfTau28_hez_final_16.ptau, hash BLAKE2b được ghim); Phase 2 gồm 2 lượt đóng góp (entropy qua stdin), một beacon ngẫu nhiên sinh cục bộ, rồi `snarkjs zkey verify`. Vì mọi lượt chạy trên một máy nên đây là nghi thức **demo**; triển khai thật cần nhiều bên độc lập (chỉ cần một bên trung thực là khóa an toàn).



Hình 11: Workflow tạo proof: hash video của cover và message tạo binding; secret camera và đường Merkle tạo witness; proof được nén rồi đóng gói vào payload.

14.7 Witness là gì?

Witness là tập giá trị làm cho circuit thỏa mãn: secret camera, đường Merkle và các giá trị trung gian khi circuit tính Poseidon và dừng lại root. Nếu không biết secret của một lá trong cây, không thể tạo witness hợp lệ cho root đã công khai.

Witness được tạo bằng `node generate_witness.js camera_video.wasm input.json witness.wtns` trong một thư mục tạm dưới `circuits/build/`. Input JSON chứa secret camera, nên các file tạm (input, witness, proof, public) được xóa trong khối `finally`; thư mục tạm cũ hơn 1 giờ do lần chạy bị dừng đột ngột được dọn khi khởi tạo bridge.

14.8 Proof JSON và proof bytes khác nhau thế nào?

`snarkjs` tạo proof ở dạng JSON gồm `pi_a` ($A \in G_1$), `pi_b` ($B \in G_2$), `pi_c` ($C \in G_1$), `protocol = groth16`, `curve = bn128` (tức BN254). Hệ thống nén proof thành 129 byte bằng `proof_to_bytes`:

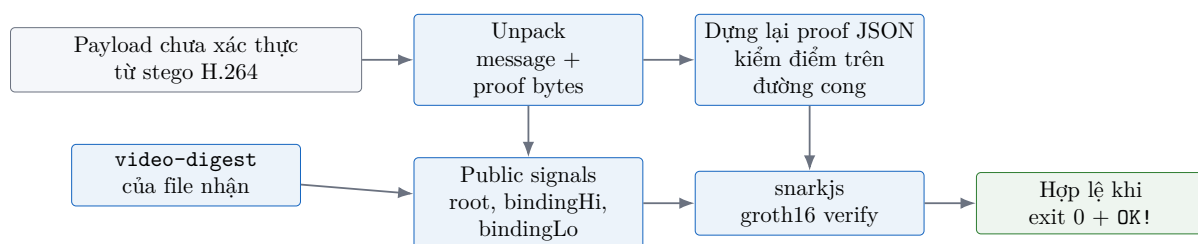
Thành phần	Kích thước
pi_a.x	32 byte
pi_b.x[0]	32 byte
pi_b.x[1]	32 byte
pi_c.x	32 byte
Byte cờ: 3 bit chẵn/lẻ của tọa độ y (A, B, C)	1 byte
Tổng	129 byte

Khi giải nén, `bytes_to_proof` kiểm tra: đúng 129 byte, không có bit cờ lạ, tọa độ nhỏ hơn p , và điểm nằm trên đường cong. Dữ liệu hỏng $\rightarrow$ `ValueError` $\rightarrow$ verifier trả `valid = False`.

14.9 Verify sau khi extract diễn ra thế nào?

Hàm `verify_payload(bridge, root, payload, native_cli=..., stego=...)` trong `src/camera_proof.py`:

1. Trích xuất mù (mục trước) để có payload chưa xác thực.
2. `unpack_payload`: kiểm format 0x01, đọc mode và độ dài, tách message và 129 byte proof.
3. Dựng lại proof JSON bằng `bytes_to_proof()` (có kiểm điểm).
4. Tính video-digest của **chính file nhận được** (cùng `frame_bits`, cap và khóa K), rồi dựng lại binding từ mode, digest và message.
5. Tạo 3 public signal [`root`, `bindingHi`, `bindingLo`] với root tin cậy.
6. Gọi `snarkjs groth16 verify` với `camera_video_vkey.json`; chỉ chấp nhận khi **mã thoát 0 và có dòng OK!**, và chỉ coi là gắn video (`video_bound`) khi `mode = 0`.



Hình 12: Workflow verify: hệ thống dựng lại proof, tính hash video của file nhận, dựng public signals với root tin cậy rồi gọi Groth16 verifier.

14.10 Proof chứng minh gì, không chứng minh gì

- Chứng minh: một camera có khóa trong sổ đăng ký đã xác nhận đúng video (mọi bit; các bit carrier được bảo vệ gián tiếp qua payload) và message; không lộ camera nào; bên kiểm chứng không cần và không biết secret camera, nên không tự làm giả được proof.
- Hệ quả đã kiểm bằng ma trận tấn công (11/11 đúng kỳ vọng): chép payload sang video khác, cắt video, sửa frame sau, lật bit dấu ngoài khung, mã hóa lại, đổi message, camera ngoài sổ đăng ký, sai khóa — đều bị từ chối.
- Không chứng minh: camera thật sự quay cảnh đó (ví dụ quay lại màn hình); luồng live (`mode = 1`) chỉ gắn message; setup Groth16 Phase 2 là nghi thức demo.

Thành phần	Trách nhiệm
ZKP layer	Chứng minh camera trong sổ đăng ký xác nhận binding (video + message) mà không lộ secret camera hay danh tính camera.
Kênh có khóa	Chọn vị trí theo khóa, làm trắng bit, loại sớm khóa sai qua header khung.
Lỗi media	Đặt và đọc bit dấu mà bitstream vẫn hợp lệ; tính hash video.
Verifier	Kiểm proof Groth16 với root tin cậy và hash video của file nhận trước khi dùng message.

15 Luồng Thực Tế: CLI, Dịch Vụ HTTP/WebSocket Và Demo

15.1 Luồng đầu-cuối

Gửi:

1. Encode video về Baseline/CAVLC.
2. `src.camera_proof.build_payload`: video-digest của cover → binding → Groth16 proof (secret camera + đường Merkle) → `proof_to_bytes` (129 B) → `pack_payload`.
3. `zkstego_blind_bits embed-stream-auth-stdin` (stdin: khóa K , payload hex).

Nhận (cần stego, khóa K , root sổ đăng ký, verification key; không cần secret camera):

1. `zkstego_blind_bits extract-stream-auth` → payload chưa xác thực.
2. Hàm `src.camera_proof.verify_payload` gọi lần lượt `unpack_payload` → `bytes_to_proof` → video-digest của file nhận → binding → Groth16 verify với root tin cậy; chỉ khi hợp lệ (và `video_bound`) mới dùng message.

15.2 Các lệnh native

Lệnh	Mục đích
<code>zkstego_blind_bits embed-stream-auth-stdin, extract-stream-auth</code>	Khung v3, lịch theo đoạn IDR, đọc/ghi file. Hậu tố <code>-auth</code> nay nghĩa là “có khóa”; khung không có MAC.
<code>zkstego_blind_bits video-digest</code>	Hash ràng buộc video (khóa K qua stdin để định vị bit carrier); cover và stego cho cùng giá trị.
<code>zkstego_blind_bits measure-live-capacity-stdin</code>	Đọc Annex-B từ stdin, in JSON dung lượng ứng viên.
<code>zkstego_blind_bits embed-live-auth-stdin, extract-live-auth-stdin</code>	Như chế độ đoạn nhưng video vào/ra qua stdin/stdout.
<code>zkstego_inspect <video></code>	Công cụ soi duy nhất: JSON NAL/slice/ứng viên.
<code>zkstego_inspect <video> --summary</code>	Tóm tắt dễ đọc.
<code>zkstego_inspect <video> --macroblock NAL MB</code>	Header mọi MB của slice và từng phần tử CAVLC của một MB.
<code>zkstego_inspect <video> --segments MAX_BITS</code>	Ứng viên theo đoạn (<code>segments-1</code>).

15.3 Dịch vụ HTTP/WebSocket

Một ứng dụng: `src.api.native_handlers:create_native_app`, dựng trên lõi job dùng chung `src.api.app.create_app`.

Endpoint	Mô tả
GET /health	Công khai.
POST /api/v1/jobs/embed	Upload <code>.h264</code> + <code>message_b64</code> (các byte được nhúng nguyên văn làm payload; payload camera có proof do phía camera dựng sẵn) + <code>secret_key_b64</code> (khóa K) → job id.
POST /api/v1/jobs/extract	Upload stego + khóa → job id; payload trả về kèm "verified": false.
POST /api/v1/jobs/verify	Trích mù → unpack → hash video của file tải lên → Groth16 verify bắt buộc với root sổ đăng ký (ZK_STEGO_CAMERA_REGISTRY; thiếu thì job lỗi). succeeded kèm message và video_bound chỉ khi proof đúng; ngược lại rejected (payload_not_found, malformed_proof_payload, video_binding_required, video_digest_unavailable, proof_invalid); lỗi hạ tầng (thiếu sổ đăng ký, khóa, Node.js) → failed.
GET /api/v1/jobs/{id}	Trạng thái job.
GET /api/v1/jobs/{id}/artifact	Tải kết quả một lần, hết hạn sau 10 phút.
WS /api/v1/stream	Khung JSON start, rồi các chunk Annex-B nhị phân.

Bảo vệ: Bearer token `ZK_STEGO_API_TOKEN` bắt buộc, tối thiểu 32 ký tự, được kiểm **trước khi đọc body**; 10 lần xác thực sai từ một IP trong 60 s → 429 kèm `Retry-After` (cả HTTP lẫn WebSocket); tối đa 4 upload đồng thời; hàng đợi job có giới hạn; khóa đi vào tiến trình native qua stdin, không qua shell. Trạng thái job: `queued`, `running`, `succeeded`, `rejected` (proof không hợp lệ — không được coi là thành công), `failed`.

15.4 Quan sát bằng demo

Demo duy nhất là `demo/terminal_demo.py`:

```
py -3.12 demo/terminal_demo.py --setup
py -3.12 demo/terminal_demo.py --auto --message "Xin chào"
py -3.12 demo/terminal_demo.py --auto --brief --no-service
py -3.12 demo/terminal_demo.py --auto --frames 48 --gop 12
```

Các bước liên quan trực tiếp đến tài liệu này: bên trong `FFmpeg/libx264` (log verbose, thống kê x264); bảng NAL, EBSP→RBSP và header do `trace_headers` giải; sổ đăng ký 4 camera, hash video của cover, binding và proof; dung lượng theo đoạn và lịch (khóa con HKDF, khung v3 không MAC, bảng IDR nào mang khoảng bit khung nào, score HMAC, bit khung $\oplus$ keystream); nhúng native; giải phẫu một khối (đối chiếu từng mẫu với ảnh `FFmpeg -skip_loop_filter all`); bit nhúng nằm ở đâu; verifier chỉ với stego, khóa K , root sổ đăng ký và verification key, Groth16 bắt buộc; các ca từ chối (đổi message, proof sai/thiếu, sai khóa, lật một bit carrier trong file, cắt frame cuối, camera ngoài sổ đăng ký); job HTTP và luồng WebSocket. `--brief` bỏ các bước giải thích sâu, `--no-service` bỏ hai bước dịch vụ. Mọi kiểm tra in `[PASS]/[FAIL]`; một `[FAIL]` dừng demo với mã thoát 1.

16 Vì Sao Chỉ IDR Mang Dữ Liệu?

Parser native chỉ giải slice IDR (I-slice). Với `-g 1` mọi frame là IDR, nên mọi frame mang ứng viên; đây là cấu hình mặc định của demo và fixture kiểm thử. Lý do:

- residual của I-slice chỉ phụ thuộc dự đoán intra trong cùng frame;
- mỗi IDR cùng SPS/PPS tạo thành một đoạn độc lập, nên lịch có thể tính theo từng đoạn, phù hợp cho cả file lẫn luồng live;
- CAVLC Baseline có cấu trúc trailing-one phù hợp với kênh dấu.

Với $GOP > 1$ (ví dụ `--gop 12 --frames 48`), hệ thống vẫn chạy nhưng P-frame không có ứng viên, nên dung lượng chỉ đến từ IDR; thay đổi trong IDR còn lan sang các P-frame tham chiếu nó.

17 Tóm Tắt Vai Trò Của Các Thành Phần

Đường dẫn	Vai trò
<code>native/include/zkstego/cavlc_stream.hpp</code> , <code>native/src/cavlc_stream.cpp</code>	Parser H.264/CAVLC, ứng viên, HKDF, lịch HMAC, khung v3, encoder/decoder theo đoạn, hash video.
<code>native/tools/blind_bits.cpp</code>	CLI nhúng/trích <code>zkstego_blind_bits</code> .
<code>native/tools/inspect.cpp</code>	Công cụ soi duy nhất <code>zkstego_inspect</code> .
<code>native/tests/cavlc_stream_tests.cpp</code>	CTest native.
<code>src/native_blind_contract.py</code>	Hợp đồng lịch/khung bằng Python (tham chiếu), <code>segment_schedule</code> .
<code>src/zk_proof.py</code> , <code>src/camera_proof.py</code> , <code>circuits/camera_video.circom</code>	Payload, proof camera (sổ đăng ký + ràng buộc video), nén, xác minh.
<code>src/camera_registry.py</code> , <code>src/poseidon.py</code> , <code>src/video_binding.py</code> , <code>src/zk_setup.py</code>	Sổ đăng ký Merkle Poseidon, hash video + binding, setup Groth16.
<code>src/api/app.py</code> , <code>src/api/native_handlers.py</code>	Lỗi job dùng chung + dịch vụ HTTP/WebSocket.
<code>src/realtime_cavlc.py</code>	Bộ lập lịch realtime theo đoạn.
<code>src/h264_tables.py</code>	Bảng VLC CAVLC chuẩn (cho phần giải thích của demo).
<code>demo/terminal_demo.py</code> và <code>deep_trace</code> , <code>h264_explain</code> , <code>segment_plan</code> , <code>negative_cases</code> , <code>service_steps</code> , <code>native_io</code>	Demo duy nhất.
<code>src/runtest/</code>	Bộ test <code>run_all.py</code> và <code>prepare_fixtures.py</code> .

Lịch sử: trước 03/10/2026 repository còn một pipeline Python song song (`src/embedder.py`, `src/verifier*.py`, `src/core/`, `src/bitstream/`, chaos map, file phụ `positions.json`, benchmark SEC1–SEC10). Nó đã bị gỡ (vẫn xem được trong lịch sử git) và không còn là bằng chứng hiện hành.

18 Kiểm Thử, Hiệu Năng Và Giới Hạn

18.1 Kiểm thử

Ngày 04/10/2026, `py -3.12 src/runtest/run_all.py` qua **142/142 trên 12 phase** (Windows, fixture từ `prepare_fixtures.py`), CTest native 1/1. Các phase: 1 `test_zk_proof` (định dạng proof, payload, vector Poseidon, sổ đăng ký, binding, đối chiếu mạch, Groth16 thật); 2 `native_blind_contract`; 3 `native_cli_fixture` (kể cả hash video native = Python, cover = stego); 4 `h264_tables`; 5 `service_api`; 6 `native_http_channel` (proof gắn video được chấp nhận, phát lại/cắt video bị từ chối, WebSocket); 7 `manifest_security`; 8 `realtime_scheduler`;

9 benchmark_recorder; 10 trust_interfaces; 11 demo_h264_explain; 12 demo_smoke. E2E camera vật lý (--hardware, H1) không nằm trong lần chạy này. Mã thoát run_all: 0 = tất cả qua, 1 = có lỗi, 2 = có ca bị bỏ qua.

18.2 Hiệu năng (lần ghi 04/10/2026)

Đo trên i7-12700H, Windows 11, giao thức v3 + proof camera.

Nguồn: benchmark/NEW_BENCHMARKS.md, mục “Recorded run”.

Hạng mục	Kết quả
Bộ media (run 20261004T142448Z_1db00a)	Chuyển đổi 9/9, media 27/27; mọi proof gắn video; mỗi case một khóa mới; sổ đăng ký 8 camera.
E2E 5/5 (clip CIF 300 frame, trung vị)	Hash video cover 0,51 s, Groth16 prove 1,77 s, nhúng 0,50 s, decode nghiêm ngặt 0,15 s, trích 0,53 s, verify (hash stego + Groth16) 1,45 s; tổng phía gửi 2,78 s, phía nhận 1,99 s.
Ma trận tấn công	11/11 đúng kỳ vọng.
Bộ ZKP	Groth16 prove 1,5–1,8 s, verify 0,9–1,0 s, proof 129 B; PLONK prove khoảng 22 s, verify 0,90–0,94 s, proof khoảng 2,25 KB JSON. Mạch SHA-256 cũ cần 3,5–4,4 s để prove.
Chi phí native (352x288 / 640x480 / 1280x960)	Quét dung lượng mỗi IDR 52 / 220 / 770 ms; nhúng mỗi IDR mang dữ liệu 61 / 245 / 891 ms; hash video (26 IDR carrier) 1,6 / 6,4 / 22,6 s. Parse, không phải Groth16, giới hạn thông lượng.
Chất lượng	PSNR-Y thấp nhất mỗi case 44,03–58,68 dB, 0/27 dưới 40 dB; thống kê dấu $ z \leq 0,22$.

18.3 Kết quả bảo mật và giới hạn còn lại

Ma trận tấn công (benchmark/results/e2e_new.json): bản gốc được chấp nhận; sai khóa → payload_not_found; lật bit dấu trong lịch → malformed_proof_payload; lật bit dấu ngoài lịch trong IDR mang dữ liệu, lật bit ở frame sau, cắt nửa video, đổi message, camera ngoài sổ đăng ký, chép payload sang video khác (akiyo, cùng khóa) → proof_invalid; bỏ IDR đầu, mã hóa lại → payload_not_found.

- **Luồng live** chỉ gắn được message (mode = 1); job verify từ chối trừ khi bật ZK_STEGO_ALLOW_MESSAGE_ONLY_PROOFS=1.
- **Setup Groth16**: Phase 2 là nghi thức demo trên một máy.
- **Proof không chứng minh** camera thật sự quay cảnh đó (ví dụ quay lại màn hình).
- **Hash video phụ thuộc khóa giấu tin**: chỉ người có K mới kiểm chứng được (bên kiểm chứng vốn cần K để trích).
- **Làm trắng không có nonce**: cùng K thì keystream được dùng lại.
- **Chưa chứng minh tính khó phát hiện**: chưa đo steganalysis.
- **Realtime chưa được chấp nhận**: E2E camera thật (run 24, ngày 24/09/2026, giao thức v1) đo được 7,608 FPS so với cổng 30 FPS; số đo này có trước đợt tăng tốc native ngày 02/10/2026 và chưa được đo lại. Chưa có kết quả trên thiết bị edge.
- **Đầu vào**: chỉ Baseline, CAVLC, progressive, 4:2:0, một slice/frame bắt đầu ở MB 0, không I_PCM, một SPS/PPS id; chỉ IDR mang dữ liệu.

- **Mong manh theo thiết kế:** mọi lần mã hóa lại đều xóa dữ liệu.

Kế hoạch xử lý các điểm trên: `future_plan.md`.

19 Điểm Cần Nhớ

Điểm tinh túy của hệ thống không phải là “đổi hệ số”. Điểm tinh túy là:

Chỉ chạm vào một bit mà cú pháp CAVLC cho phép đổi mà không ảnh hưởng gì khác, để khóa bí mật quyết định bit nào mang dữ liệu, và bất mọi thứ đọc ra phải qua một proof Groth16 gắn với chính video đó trước khi được tin.

Các thuật ngữ quan trọng nhất:

- **Ứng viên:** bit đầu của trailing-one đầu tiên trong một khối residual của slice IDR; một ứng viên mỗi khối.
- **Bất biến độ dài:** lật dấu không đổi TotalCoeff/TrailingOnes, nên không đổi nC, độ dài bit RBSP hay cú pháp phía sau.
- **Lịch có khóa theo đoạn:** HMAC(schedule_key, định danh) xếp hạng ứng viên trong mỗi đoạn SPS+PPS+IDR, tối đa max_bits_per_IDR bit/đoạn.
- **Khung v3 và làm trắng:** [0x03][len][payload] (không MAC), XOR với keystream HMAC.
- **Trích xuất mù:** không cần video gốc hay file phụ.
- **Hash video:** SHA-256 của mọi NAL với đúng các bit carrier đặt về 0; cover và stego cho cùng giá trị, mọi sửa đổi khác làm nó đổi.
- **Groth16 bắt buộc:** tìm thấy khung chưa đủ; proof camera phải verify với root sổ đăng ký và hash video của file nhận (exit 0 + OK!).

20 Kết Luận

Hệ thống xử lý video theo hướng rất cụ thể:

H.264 Baseline → đoạn IDR → ứng viên dấu T1 → lịch HMAC → bit khung ⊕
keystream → stego H.264

Nó không giấu tin bằng cách sửa pixel. Nó giấu tin bằng cách đặt bit dấu của một số hệ số trailing-one do khóa chọn, trong một lõi C++ duy nhất. Tính hợp lệ của bitstream đến từ cấu tạo của CAVLC; việc chỉ người có khóa tìm được dữ liệu đến từ HKDF, lịch HMAC và làm trắng; còn tính xác thực — một camera đã đăng ký xác nhận đúng video và message này — được chứng minh bằng Groth16, và bên kiểm chứng không cần secret camera. Những gì hệ thống *chưa* làm — gắn video cho luồng live, nghi thức setup nhiều bên, đạt realtime trên camera thật — được ghi rõ ở trên và trong `future_plan.md`.